
Web Development and Design

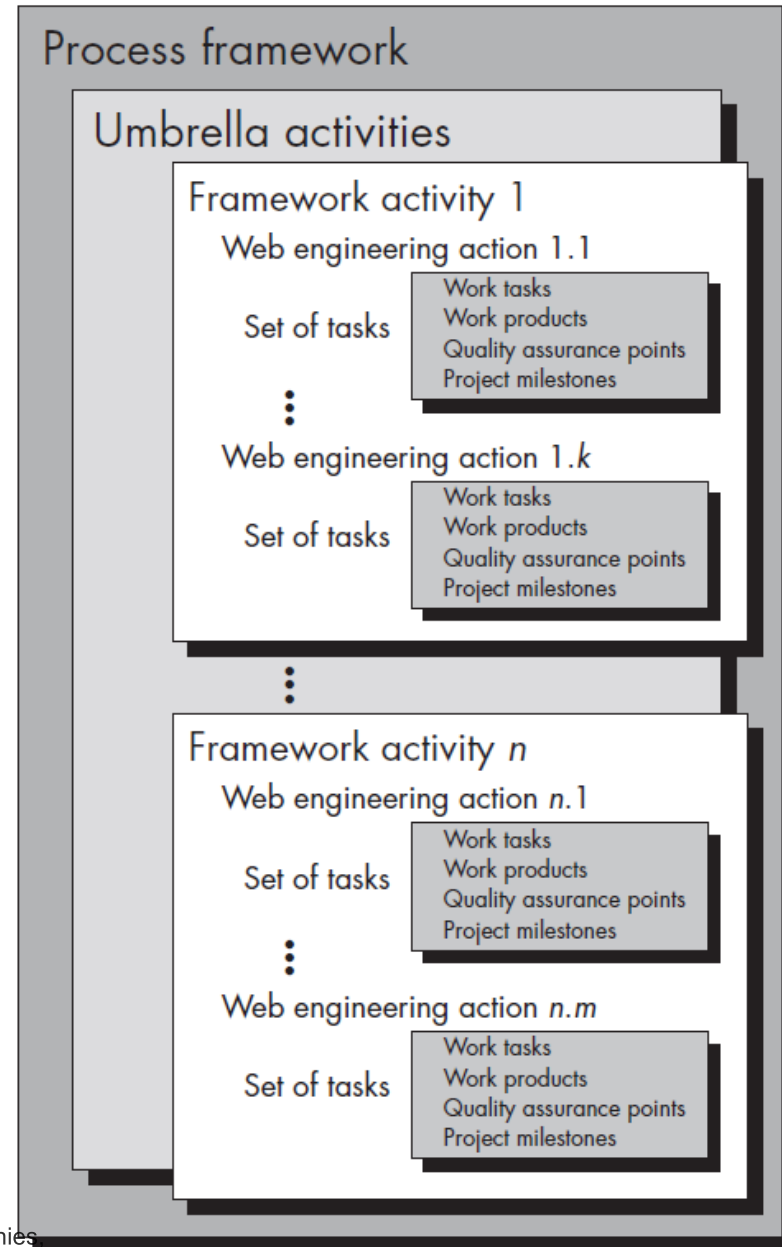
Lecture 02: A Web Engineering Process

What is a Web Engineering Framework?

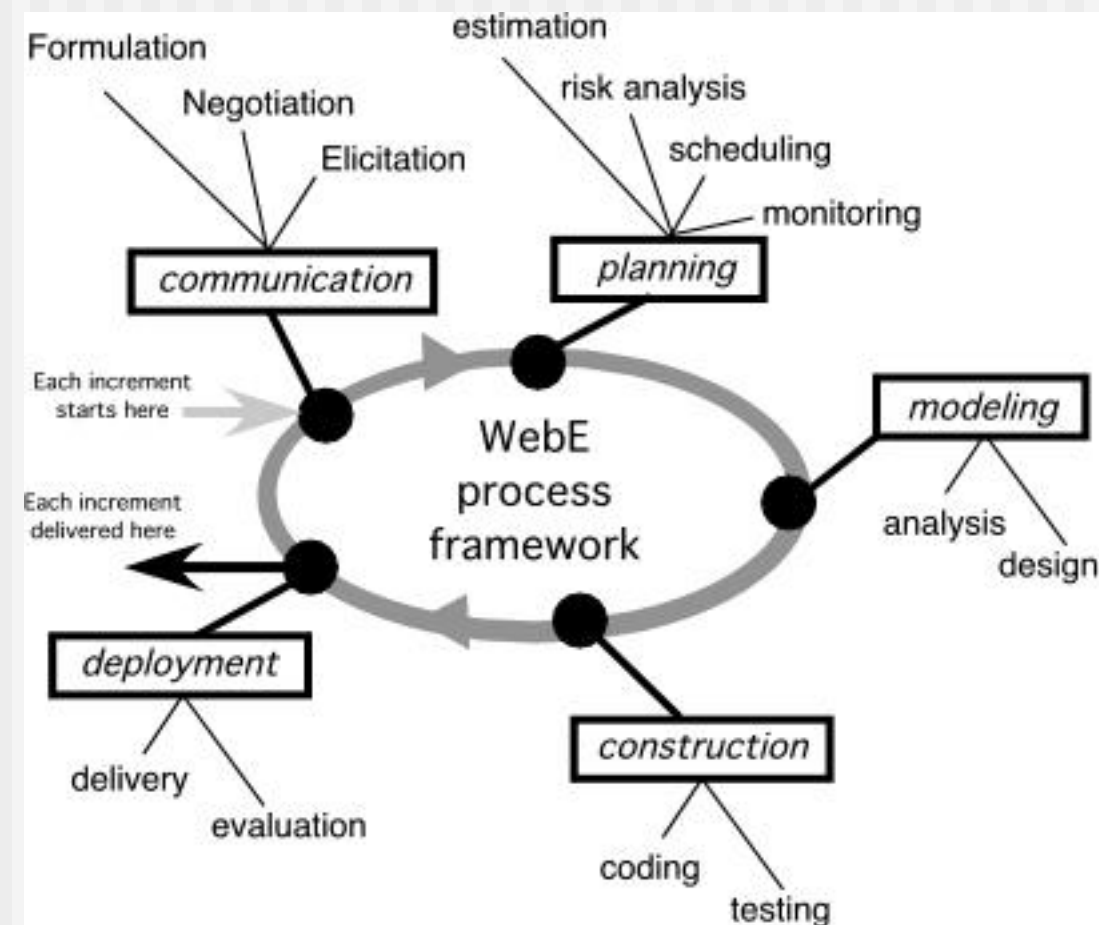
- A framework is a set of activities that will *always* be performed for *every* Web engineering project – though the nature of the activities might vary to suit the project.
- Each framework activity is composed of a set of actions
- Actions encompass
 - work tasks
 - work products
 - quality assurance points, and
 - project milestones
- A framework also has a set of “umbrella activities”

A Generic Framework

WebE process



WebE Process Activities & Actions



The WebE Framework: Activities

- **Communication.** Involves heavy interaction and collaboration with the customer (and other stakeholders) and encompasses requirements gathering and other related activities.
- **Planning.** Establishes an incremental plan for the WebE work.
- **Modeling.** Encompasses the creation of models that assist the developer and the customer to better understand WebApp requirements and the design

The WebE Framework: Activities

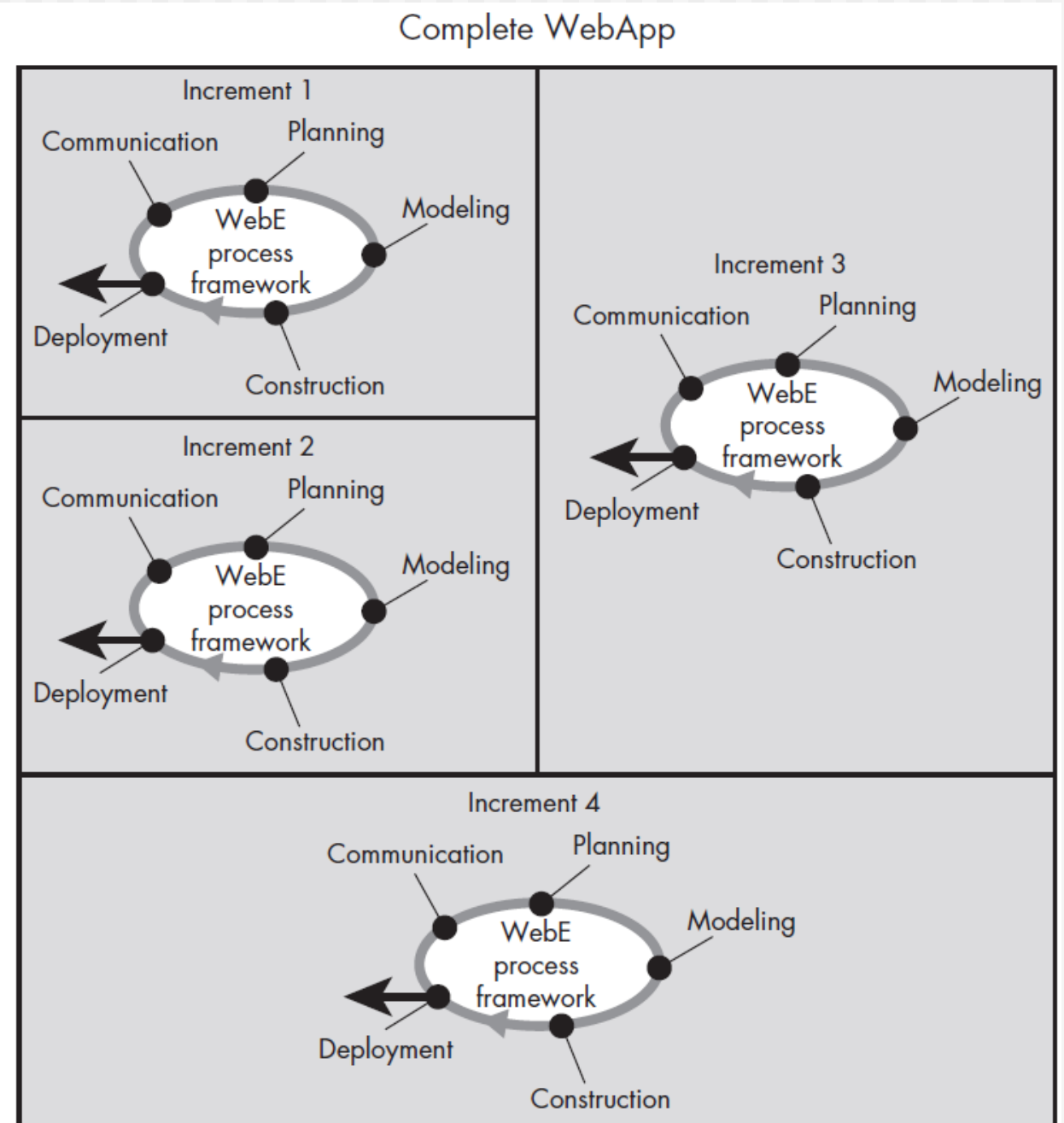
- **Construction.** Combines both the generation of HTML, XML, Java, and similar code with testing that is required to uncover errors in the code.
- **Deployment.** Delivers a WebApp increment to the customer who evaluates it and provides feedback based on the evaluation.

The WebE Process

- The process must be agile and adaptable, but it must also be *incremental*
- Why incremental?
 - Requirements evolve over time
 - Changes will occur frequently (and always at inconvenient times)
 - Timelines are short
- Incremental delivery allows you to manage this change!

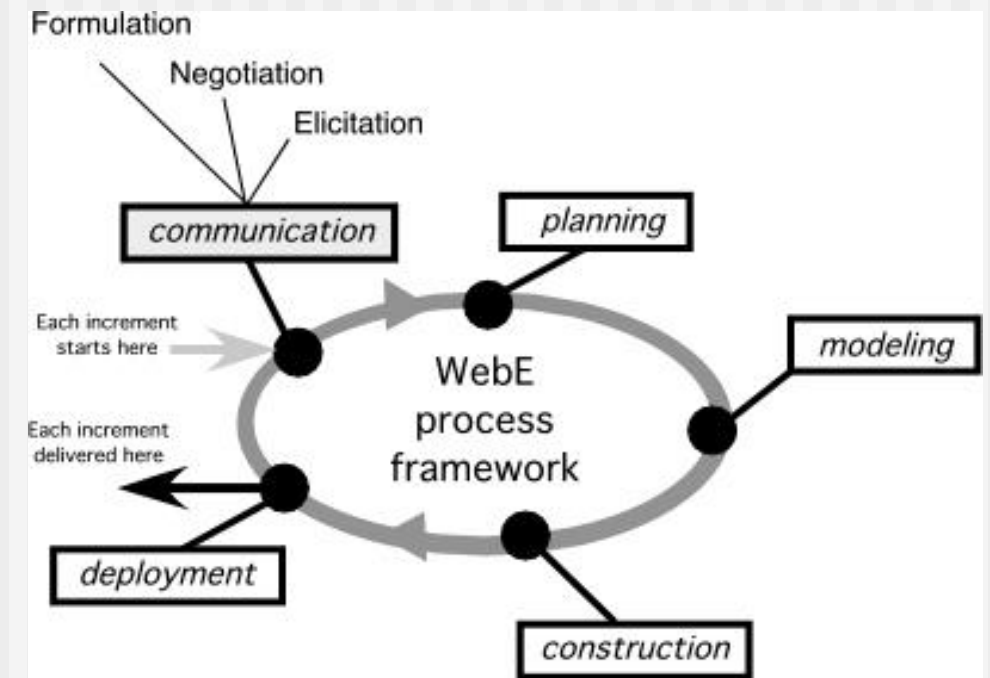
Incremental Delivery

Repeat the development cycle for each increment!



WebE Process: Communication

- *Understand the problem before you begin to solve it, and be sure that the solution you conceive is one that people really want*
- To do this, you'll need to:
 - *Formulate*
 - *Elicitate*
 - *Negotiate*



Who Should You Communicate With?

- A stakeholder can be defined as “anyone who benefits in a direct or indirect way from the system which is being developed”
- In general, stakeholders are drawn from the following categories:
 - business managers,
 - product managers,
 - marketing people,
 - Internal and external customers,
 - end users,
 - consultants, product engineers,
 - Web engineers,
 - and support and maintenance staff.

Formulation

- Focuses on defining the project needs and scope
 - begins with the **identification of a business need**
 - moves into a **description of WebApp objectives**
 - defines **major WebApp features**, and
 - establishes a **basis for the elicitation** action that follows.
- **Why is Formulation Important?**
 - **Common Goals:** It ensures everyone understands the project's purpose and agrees on what needs to be built.
 - **Clear Scope:** It defines the boundaries of the project

What Questions Do We Ask?

- What is the main motivation (business need) for the WebApp?
- What are the objectives that the WebApp must fulfill?
- Who will use the WebApp?
- Note that:
 - Every stakeholder has a different view of the WebApp, achieves different benefits when the WebApp is successfully deployed, and is open to different risks if the development effort should fail.
 - As information from multiple viewpoints is collected, emerging **requirements may be inconsistent or may conflict** with one another.
 - Your job during formulation and elicitation is to categorize all stakeholder information (including inconsistent and conflicting requirements) in a way that will set the stage for the last WebE action, *negotiation*.

Case Study (Formulation)

- It's time to meet with CPI Corporation stakeholders and begin the ***communication activity***.
- Representatives from business management, marketing, product engineering, customer service, sales, and product support are present.
- Your intent is to formulate a view of the business context for **SafeHomeAssured.com**.

Case Study (Formulation)

- ***What is the main motivation (business need) for the WebApp?***
- After some discussion, the group agrees on the following simple statement of motivation:
- **SafeHomeAssured.com will allow customers to evaluate, configure, and purchase security system products and monitoring services.**

Case Study (Formulation)

- *What are the objectives that the WebApp must fulfill?*
- various stakeholders, an answer to the second question is stated:
 - SafeHomeAssured.com will allow us to **sell directly to consumers**, thereby eliminating middleman costs and improving our profit margins. It will also allow us to increase sales by a projected 25 percent over current annual sales and will allow us to **penetrate geographic regions** where we currently do not have sales outlets. It will **reduce the need for expanding our customer service** call center by providing features that **allow a customer to access monitoring information directly**.

Case Study (Formulation)

- Finally, the third question (***Who will use the WebApp?***) is addressed and user characteristics are defined for the WebApp:
 - **Projected end users (the primary demographic) of SafeHomeAssured.com are homeowners and owners of small businesses. However, the application will also be used by CPI sales and service staff.**

Case Study (Formulation)

- These answers imply a set of specific goals for the **SafeHomeAssured.com** Web App.
- In general, two categories of goals are identified:
 - ***Informational goals*** that indicate an intention to provide specific content . and/or information for the end User.
 - ***Applicative goals*** that indicate the ability to perform some task within the WebApp.

Case Study (Formulation)

- In the context of the **SafeHomeAssured.com** WebApp, a list of **informational goals** is developed:
 - To allow the user to learn about CPI Corporation, its people, and products
 - To provide users with detailed product specifications, including technical descriptions, installation instructions, and pricing information
 - To display the security configuration (including all hardware) for the facility (house or business) that the end user defines
 - To enable a user to obtain a quote for product cost
 - To create an account database that provides account information for customers

Case Study (Formulation)

- Among the **applicative goals** for **SafeHomeAssured.com** are the following:
 - To provide tools that will enable a user to represent the layout of a “space” (i.e., house, office/retail space) that is to be protected
 - To make customized recommendations about security and monitoring products that can be used within the user space
 - To integrate forms-based SafeHomeAssured.com input with CPI’s existing customer and order entry database
 - To allow users to place an order for security hardware
 - To allow users to control monitoring equipment (e.g., cameras, microphones) within their space
 - To enable users to sign up for monitoring services
 - To allow monitoring customers to query the monitoring database about their account activity

Formulation

- Once the WebApp objective, goals, and user profiles have been developed, the formulation activity focuses on **a *statement of scope*** for the WebApp.
- **Scope** means defining ***what the website will and will not include***. It's about setting **boundaries** for the project.
- Example of Scope Definition
 - **In Scope:** Online ordering system with secure payment gateway.
 - **Out of Scope:** Delivery service integration (might be considered later).
- In many cases, informational and applicative goals provide sufficient information to identify the scope of the problem.

Case Study

- The WebApp description is assembled using the information already derived from the formulation meeting. For the **SafeHomeAssured.com** WebApp, the description (a *statement of scope*) might look like:

SafeHomeAssured.com will allow CPI Corporation to market and sell security products and monitoring services directly to consumers, thereby eliminating middleman costs and improving our profit margins. To accomplish this goal, the WebApp will incorporate both content and functions that implement the following product-related features:

- End users can examine the *SafeHome* product line and request product specs.
- Users can configure a security system by representing the layout of a “space” (i.e., house, office/retail space) and then initiating functions within **SafeHomeAssured.com** to make customized recommendations about security and monitoring products that can be used within the space.
- Users can request an instant quote for product and/or system pricing.
- Users can place an order for sensors, controllers, audio, and video hardware and related infrastructure.

The WebApp will incorporate both content and functions that implement the following monitoring-related features:

- Users can sign up for monitoring services, order installation of a *SafeHome* system, and coordinate all other setup activities that will lead to a purchase of security products, their installation, and the execution of a monitoring contract with CPI.
- Contract customers can control security and monitoring equipment (e.g., cameras, microphones) within their space and use the **SafeHomeAssured.com** WebApp to acquire the output of security devices and display them for their own customers.
- Customers can query the CPI monitoring database about their account activity.

The **SafeHomeAssured.com** WebApp must be implemented so that usage is intuitive. It must have very strong security features and be available 24/7/365.

The **SafeHomeAssured.com** WebApp will also have specialized features for in-house CPI staff. These features will provide enhanced customer support and security system technical support and maintenance.

Elicitation

- The intent is to gather detailed requirement collaboratively with all stakeholders
- To do this:
 - A **meeting (either physical or virtual) is conducted** and attended by all stakeholders.
 - **Rules for preparation and participation are established.**
 - **An agenda** is suggested that is formal enough to cover all important points but informal enough to encourage the free flow of ideas.
 - **A facilitator** Keeps the meeting productive, manages discussions, and ensures everyone contributes. (can be a customer, a Web engineer, ...).
 - **A definition mechanism** (can be work sheets, flip charts, or wall stickers or an electronic bulletin board, chat room, or virtual forum) is used to Captures and organizes ideas.

Content and Functionality

- Each stakeholder has begun this work by preparing lists of **content objects** and **WebApp functions**.
 - **Content Objects:** What information should the website contain?
 - Examples: online store products, blog articles.
 - **WebApp Functions:** What should the website do? What can users do on the website?
 - Examples: Buy tickets online, leave feedback.

Content and Functionality

- Once the meeting begins these lists can be:
 - displayed on large sheets of paper pinned to the walls of the room
 - displayed on adhesive-backed sheets stuck to the walls, or
 - written on a whiteboard.
 - posted on an electronic bulletin board, at an internal website, or posted in a chat room environment for review prior to the meeting.
- At this stage, critique and debate are strictly prohibited. This is a brainstorming phase. The goal is to gather as many ideas as possible, no matter how wild or impractical they might seem initially.

Content and Functionality

- **Why is this "no critique" rule important?**
 - **Encourages Creativity:** People are more likely to share their ideas if they know they won't be immediately shot down.
 - **Generates More Ideas:** Brainstorming without criticism leads to a greater quantity and variety of ideas.
 - **Prevents Conflict:** Early critique can stifle discussion and create tension among stakeholders.

Case Study (Elicitation)

- Each stakeholder is asked to review the WebApp description before the requirements gathering meeting and make a list of “**content objects**” that are:
 - Part of the environment that surrounds the system,
 - produced by the system, and
 - used by the system to perform its functions.
- In addition, each attendee is asked to make a **list of functions** that manipulate or interact with the content objects. Finally, lists of constraints (e.g., business rules that will impact the features provided) and performance criteria (e.g., speed, accuracy, privacy, security) are also developed

Case Study (Elicitation)

- Each person develops the lists described previously. **Content objects** described for the **SafeHomeAssured.com** WebApp might include:

- Company profile
- Product overview
- Product specifications
- Installation instructions
- Product database (includes pricing, items in inventory, shipping costs, etc.)
- Quotation templates
- Order forms
- Order entry database
- Layout image of user space
- User database (includes user identification, specialized security system configuration, other user-related info)
- Monitoring request form
- Monitoring "dashboard" for a contract user (includes account information, video window for monitoring, audio port for monitoring)
- Monitoring database
- Customer service dashboard (to be defined later)
- Tech support dashboard (to be defined later)

Case Study (Elicitation)

- Each of these content objects represents a preliminary content requirement and also implies a **set of functions** that will be required to acquire, manipulate, or produce the content object.

- Provide product quotation
- Process security system order
- Process user data
- Create user profile
- Draw user space layout
- Recommend security system for layout
- Process monitoring order
- Get and display account info
- Get and display monitoring info
- Customer service functions (to be defined later)
- Tech support functions (to be defined later)

Elicitation Tasks

- *After* this initial brainstorming, the web engineer takes the output of the meeting (the lists of ideas, notes, etc.) and performs the following tasks:
 - **Define user categories, and develop descriptions for each category.**
 - **Define content and functionality using the lists each person prepared.**
 - **Consider specific constraints and performance issues.**
 - **Write user scenarios for each user class.**

User Descriptions

- What is the user's overall objective when using the WebApp?
- What is the user's background and sophistication level relative to the content and functionality of the WebApp?
- How will the user arrive at the WebApp?
- What generic WebApp characteristics does the user like and dislike?
 - Using the answers to these questions, set of **user classes** should be defined. These defined user classes are **essential** when developing "**Usage Scenarios**" (also called **user stories**). Usage scenarios describe how each type of user will interact with the WebApp to achieve their objectives.

■

What Are Usage Scenarios?

- Stakeholders are asked to play the role of a user (from a specific user category) and develop usage scenarios for one or more entries on each of the lists.
- Each usage scenario is typically one or two narrative paragraphs that **describe how an end user would apply or create a content object or interact with a WebApp function.**
- It should be noted, however, that **more formal templates** have been proposed and are **called use cases.**

Case Study (Usage Scenarios)

Owner actor might write the following narrative:

Use case: *Access camera surveillance via the Internet*

Actor: HomeOwner

Narrative: If I'm at a remote location, I can use any PC with appropriate browser software to log on to the **SafeHomeAssured.com** website. I enter my user ID and two passwords, and once I'm validated, I have access to all functionality for my installed *SafeHome* system. To access a specific camera view, I select "Surveillance" from the major function buttons displayed. I then select "Pick a camera," and the floor plan of the house is displayed. I then select the camera that I'm interested in. Alternatively, I can look at thumbnail snapshots from all cameras simultaneously by selecting "All cameras" as my viewing choice. Once I choose a camera, I select "View" and a camera view appears in a viewing window that is identified by the camera ID. If I want to switch cameras, I select "Pick a camera" and the original viewing window disappears and the floor plan of the house is displayed again. I then select the camera that I'm interested in. A new viewing window appears.

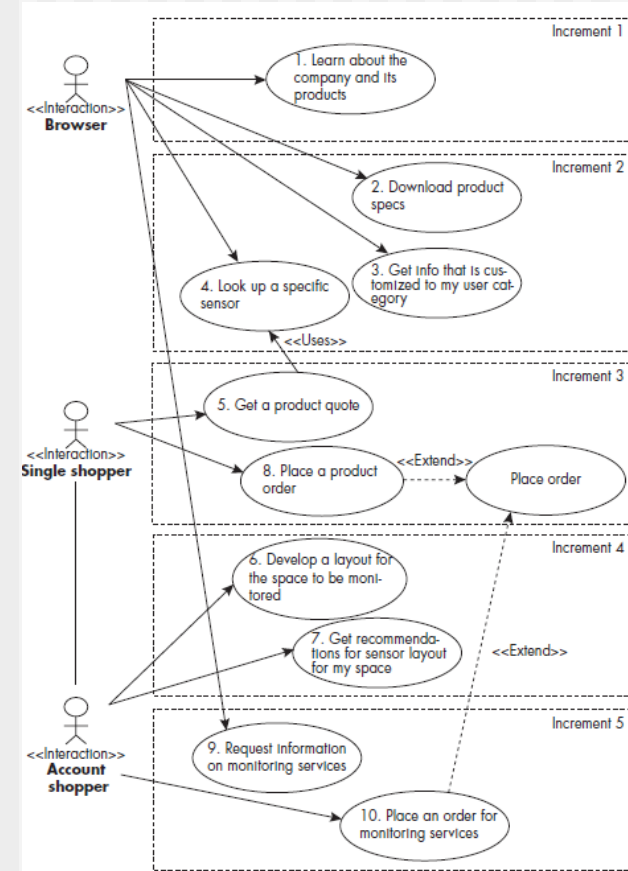
Capturing Interaction: Use Cases

- Use cases describe how a specific **user category** (called an *actor*) will interact with the WebApp to accomplish **a specific action**.
- Use cases are developed iteratively. Only those use cases necessary for the increment to be built are developed during the communication activity for the increment.
- **Use cases are similar to usage scenarios** but are typically **more structured and detailed**. They often involve diagrams and specific steps to describe the interaction.

Capturing Interaction: Use Cases

■ Use cases enable you to:

- provide the detail necessary for effective planning and modeling activities.
- help you to understand how **users perceive their interaction with the WebApp.**
- help to compartmentalize Web engineering work because they can be **organized into WebApp increments.**
- provide important guidance for those who must **test the WebApp** to ensure it meets user needs.



Case Study (Use Case)

Use case: *Access camera surveillance via the Internet*

Actor: HomeOwner

Actions:

1. The homeowner logs on to the **SafeHomeAssured.com** website.
2. The homeowner enters his or her user ID.
3. The homeowner enters two passwords.
4. The system displays all major function buttons.
5. The homeowner selects "Surveillance" from the major function buttons.
6. The homeowner selects "Pick a camera."
7. The system displays the floor plan of the house.
8. The homeowner selects a camera icon from the floor plan.
9. The homeowner selects the "View" button.
10. The system displays a viewing window that is identified by the camera ID.
11. The system displays video output within the viewing window at one frame per second.

From Use Cases to Increments

- A stack of “cards” that contains one usage scenario or use case per card
 - Each card contains the name of the use case, a brief description, and **an *effort indicator***—usually a number between 1 and 4
- The cards are:
 - shuffled into random order
 - distributed to selected stakeholders who are asked to arrange the cards into groupings that reflect how they would like content and functionality (implied by the usage scenarios) to be delivered

Case Study (Use Cases to Increments)

- As an example, we assume that the first communication activity for the **Safe-HomeAssured.com** WebApp has been completed, and stakeholders have defined a set of usage scenarios that are reflected in a stack of cards:

Card No.	Card Name	Effort Indicator
1	Learn about the company and its products	1
2	Download product specs	2
3	Get info that is customized to my user category	2
4	Look up a specific sensor	1
5	Get a product quote	2
6	Develop a layout from the space to be monitored	4
7	Get recommendations for sensor layout for my space	3
8	Place a product order	2
9	Request information on monitoring services	1
10	Place an order for monitoring services	2
11	Control sensors	3
12	Control cameras	3
13	Access camera surveillance via the Internet	4
14	Get account information	2

and so on . . .

Case Study (Use Cases to Increments)

In the case of the cards defined in the stack, the preliminary set of **SafeHome-Assured.com** increments might be:

Increment 1: Use cases reflected in card 1.

Increment 2: Use cases reflected in cards 2, 3, and 4

Increment 3: Use cases reflected in cards 5 and 8

Increment 4: Use cases reflected in cards 6 and 7⁹

Increment 5: Use cases reflected in cards 9 and 10

Increment 6: Use cases reflected in cards 11, 12, and 13

Increment 7: Use cases reflected in card 14

Constraints and Performance

- **Internal constraints** thinking about the **technical environment** in which the WebApp will reside and the **project environment** in which the WebApp will be built.
 - **Technical environment:** relate to the technology the website will use and how it interacts with other systems.(specialized database protocols, the vagaries of different Web browsers, operating system characteristics, and client-server issues)
 - **Project environment:** relate to the resources and tools available to the development team. (WebE tools, development hardware, software standards, and staff skill levels with various WebE technologies)

Constraints and Performance

- **External constraints** can impact a web application (WebApp) project. These constraints come from outside the development team and relate to the business, users, and legal environment in which the website will operate.
 - Business rules, End-User preferences, security demands, privacy issues, run-time performance, interoperability requirements, legal restrictions, and government regulations are but a few of possible external constraints

Negotiation

- Ideally, requirements are defined in sufficient detail to proceed.
- BUT, in reality, requirements are often contradictory or infeasible (within the context of real-world constraints, such as cost or time).
- Negotiation attempts to reconcile **conflicts between stakeholders**.
 - The intent is to establish a set of increment requirements that meets the needs of all the stakeholders while at the same time reflecting the real-world constraints (e.g., time, people, budget).

Negotiation

- *Recognize that it's not a competition.* To be successful, both parties have to feel they've won or achieved something. Both will have to compromise.
- *Map out a strategy.* Decide what you'd like to achieve, what the other party wants to achieve, and how you'll go about making both happen.
- *Listen actively.* Don't work on formulating your response while the other party is talking. Listen. It's likely you'll gain knowledge that will help you to better negotiate your position.

Negotiation

- *Focus on the other party's interests.* Don't take hard positions if you want to avoid conflict. By understanding the interest, the developers can propose alternative solutions that might also achieve the same goal
- *Don't let it get personal.* Focus on the problem that needs to be solved.
- *Be creative.* Don't be afraid to think outside of the box if you're at an impasse.
- *Be ready to commit.* Once an agreement has been reached; commit to it and move on. This builds trust and ensures the project moves forward.

Slide Set to accompany Web Engineering: A Practitioner's Approach

by Roger S. Pressman and David Lowe

copyright © 2009
Roger S. Pressman and David Lowe

For Education Use Only

May be reproduced ONLY for student use at the university level when used in conjunction with *Web Engineering: A Practitioner's Approach*.
Any other reproduction or use is expressly prohibited.

This presentation, slides, or hardcopy may NOT be used for short courses, industry seminars, or consulting purposes without the express written permission of the authors.

Thank You